

Why My PBDS ordered_multiset Solution TLE'd (and the Fenwick Fix)

Priyanshu Debnath · priyanshudebnath.me/blog/pbds-vs-fenwick-cache-tle/

Published 2026-07-05

My approach to Zhily and Barknights was correct from the start, but the PBDS ordered_multiset implementation TLE'd twice. This post covers how Red-Black trees and order statistics work, and why a Fenwick tree passed instead.

1 The problem

This one's from Codeforces Round 1097 (Div. 1), problem B — [Zhily and Barknights](#).

B. Zhily and Barknights
time limit per test: 4 seconds
memory limit per test: 512 megabytes

Zhily developed a game called Barknights, and she is preparing to release a major update on March 25. Specifically, she plans to add a Module to each Operator in the game, multiplying their power level by the Module's power.

After the update is released, the famous game streamer Jily will evaluate the Operators' power levels and rank them in a tier list. Whenever an earlier-released Operator is ranked higher than a later-released Operator, it will cause a wave of drama.

Unfortunately, Zhily accidentally knocked over a hot water kettle and broke her computer, which caused all the Modules to be randomly rearranged. Now Zhily wants to know the expected number of waves of drama that will be generated, but since she has to move on to the next problem, she has entrusted this task to you.

You are given two arrays a and b of n positive integers. Let b' be a permutation of array b chosen uniformly at random among all $n!$ possible permutations. Define $c_i = a_i \cdot b'_i$ for $1 \leq i \leq n$.

Find the expected number of inversions* of array c .

*An inversion in array c is a pair of indices (i, j) such that $1 \leq i < j \leq n$ and $c_i > c_j$.

Input
Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2000$) — the length of the arrays a and b .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the array a .

The third line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the array b .

It is guaranteed that the sum of n over all test cases does not exceed 2000.

Output
For each test case, output the expected number of inversions in c modulo 998244353.

Formally, let $M = 998244353$. It can be shown that the exact answer can be expressed as an irreducible fraction $\frac{p}{q}$, where p and q are integers and $q \not\equiv 0 \pmod{M}$. Output the integer equal to $p \cdot q^{-1} \pmod{M}$. In other words, output such an integer x that $0 \leq x < M$ and $x \cdot q \equiv p \pmod{M}$.

Codeforces Round 1097 (Div. 1, Based on Zhily Cup 2026)

Finished

Practice

Save to List

→ **Virtual participation**

Virtual contest is a way to take part in past contest, as close as possible to participation on time. It is supported only ICPC mode for virtual contests. If you've seen these problems, a virtual contest is not for you - solve these problems in the archive. If you just want to solve some problem from a contest, a virtual contest is not for you - solve this problem in the archive. Never use someone else's code, read the tutorials or communicate with other person during a virtual contest.

Start virtual contest

→ **Clone Contest to Mashup**

You can clone this contest to a mashup.

Clone Contest

→ **Submit?**

Language: GNU G++23 14.2 (64 bit, msvc)

Choose file: Choose File No file chosen

Problem statement

The gist: you're given two arrays a and b of length n . You take a uniformly random permutation b' of b , define $c_i = a_i \cdot b'_i$, and you need the **expected number of inversions** in c , modulo 998244353. Constraints: $n \leq 2000$, but $\sum n \leq 2000$ across test cases, and $a_i, b_i \leq 10^9$.

The math for this problem was correct from my first submission. What was wrong was the data structure I used to implement it — that choice cost me two TLEs before I worked out exactly why, and by how much.

2 The idea (short version)

Expected inversions is a linearity-of-expectation problem. For a fixed pair of positions $i < j$, you average over all ways to assign b -values to them. Instead of thinking permutation-by-permutation, you can restructure the sum so that you sweep i from left to right, and for each i , you compare $a_i \cdot b_j$ against *every* $a_k \cdot b_l$ pair seen so far (all inserted incrementally) — using a Fenwick tree / order-statistics structure to count how many prior values are bigger or smaller in $O(\log n)$ instead of $O(n)$ per comparison.

Concretely, my final (accepted) solution keeps two structures:

- `prodTree` — holds every product $a_i \cdot b_j$ seen for indices processed so far
- `aTree` — holds every raw a_i seen so far

and for each new i , before inserting it, it queries: "how many things bigger than my current candidate are already in here?" That count feeds directly into the running inversion total.

I won't paste the whole solve function in one block. I'll go through it piece by piece further down, after covering the theory behind the data structure, since that's what actually explains the TLE.

3 Attempt #1: PBDS `ordered_multiset`

My first working version used GNU's Policy-Based Data Structures — specifically the `tree` container with `tree_order_statistics_node_update`, aka the thing every competitive programmer calls an "order statistics tree" or `ordered_set` / `ordered_multiset`.

```
1 typedef tree<
2     pair<int, int>,
3     null_type,
4     less<pair<int, int>>,
5     rb_tree_tag,
6     tree_order_statistics_node_update>
7 ordered_multiset;
```

It compiled, gave the right answer on the sample cases, and I submitted it.

```
1 381028293 TLE on test 3 4000 ms 145200 KB
2 381028795 TLE on test 3 4000 ms 157200 KB
```

<u>381028795</u>	Jul/02/2026 16:29	Time limit exceeded on test 3
<u>381028293</u>	Jul/02/2026 16:25	Time limit exceeded on test 3

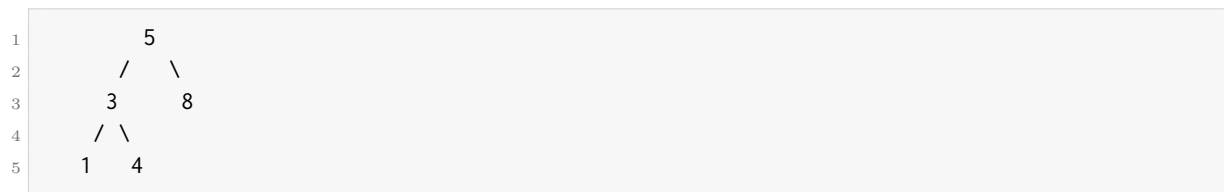
Rejected submissions

Two attempts, both failing on the same test. At that point I didn't think much, later saw the math behind it.

To understand where the time was going, it helps to understand what `ordered_multiset` actually is under the hood. So let's build up a Red-Black tree from the basics.

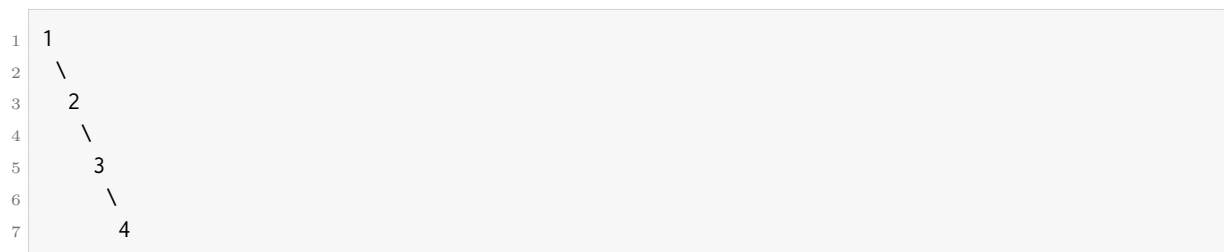
4 Part 1 — Why a plain BST isn't enough

A Binary Search Tree is a tree where, for every node, everything in the left subtree is smaller and everything in the right subtree is bigger.



Search, insert, and delete all walk down from the root, so their cost is proportional to the *height* of the tree. If you're lucky and the tree stays bushy, height is about $\log_2 n$, and everything is fast.

But if you insert values in sorted order — 1, 2, 3, 4, 5 — the tree degenerates into a straight line:



Now height equals n , not $\log n$, and every operation costs $O(n)$. This is the reason self-balancing trees exist: they force the height to stay logarithmic regardless of insertion order.

5 Part 2 — The Red-Black invariants

A Red-Black tree colors every node red or black and enforces five rules:

1. Every node is red or black.
2. The root is black.
3. Every null leaf is treated as black.
4. **A red node never has a red child** — no two reds in a row on any path.
5. **Every path from a node to any of its null descendants passes through the same number of black nodes** (its "black-height").

Rule 5 is the one doing the real work. It doesn't force every path to be the *same length* — it forces the number of *black* nodes on every path to match. Combined with rule 4 (reds can't stack), this caps how much longer any path can be than any other: at most a factor of 2.

6 The proof that height stays $O(\log n)$

Define $\text{bh}(x)$ as the black-height of node x : the number of black nodes from x down to a null leaf (not counting x itself).

Lemma. A subtree rooted at x contains at least $2^{\text{bh}(x)} - 1$ internal nodes.

Proof, by induction on subtree height.

Base case: x is a null leaf. $\text{bh}(x) = 0$, subtree contains $2^0 - 1 = 0$ nodes. True.

Inductive step: each child of x has black-height either $\text{bh}(x)$ (if the child is red) or $\text{bh}(x) - 1$ (if the child is black). Either way, by the inductive hypothesis each child's subtree has at least $2^{\text{bh}(x)-1} - 1$ nodes, so:

$$\text{size}(x) \geq 2 \left(2^{\text{bh}(x)-1} - 1 \right) + 1 = 2^{\text{bh}(x)} - 1$$

■

Now, because rule 4 forbids two reds in a row, at least half the nodes on any root-to-leaf path are black, so if h is the tree's height:

$$\text{bh}(\text{root}) \geq h/2$$

Plugging into the Lemma with $n = \text{total nodes}$:

$$n \geq 2^{\text{bh}(\text{root})} - 1 \geq 2^{h/2} - 1$$

$$h \leq 2 \log_2(n + 1)$$

That's the guarantee. However you insert data, height never exceeds roughly twice $\log_2 n$.

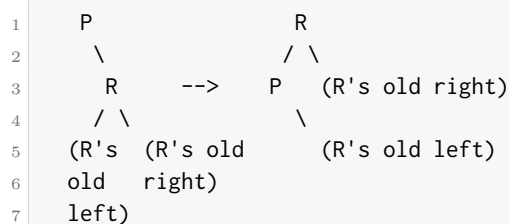
7 Part 3 — Fixing violations: rotations

When you insert a new node, it always starts **red** (inserting black would instantly break rule 5 on one path; inserting red might break rule 4, which is locally repairable).

If the new node's parent is black, nothing broke — done. If the parent is also red, you have two reds in a row, and you fix it based on the color of the "uncle" (the parent's sibling):

Case A — uncle is red. Just recolor: parent $\rightarrow$ black, uncle $\rightarrow$ black, grandparent $\rightarrow$ red. Then recheck one level higher, since the grandparent turning red might itself clash with *its* parent.

Case B — uncle is black or missing. You need a rotation. A **left rotation** around node P with right child R : R takes P 's spot, P becomes R 's left child, and R 's old left subtree becomes P 's new right subtree.



It's three pointer reassignments — $O(1)$ work — but it locally reshuffles the shape of the tree. A right rotation is the mirror image. Depending on whether the violation is a "straight line" or a "zig-zag," you need one rotation plus a recolor, or two rotations plus a recolor.

8 A worked example

Insert 10, 20, 30, 15, 5 one at a time.

Insert 10 — tree is empty, becomes root, forced black.

```
1  10(B)
```

Insert 20 — goes right of 10, colored red. Parent is black, no violation.

```

1 10(B)
2   \
3   20(R)

```

Insert 30 — goes right of 20, red. Now 20(R) has red child 30(R) — violation. Uncle is nil (black), straight-line case → rotate left at 10.

```

1      20(B)
2     /  \
3    10(R) 30(R)

```

Insert 15 — $15 < 20$, go left to 10; $15 > 10$, becomes 10's right child, red.

```

1      20(B)
2     /  \
3    10(R) 30(R)
4       \
5       15(R)

```

10(R) has red child 15(R) — violation. Uncle (30) is **red** → Case A: recolor 10 and 30 to black, 20 to red, then re-check from 20. Since 20 is root, force it back to black.

```

1      20(B)
2     /  \
3    10(B) 30(B)
4       \
5       15(R)

```

Insert 5 — $5 < 20$ → left to 10; $5 < 10$ → becomes 10's left child, red. Parent (10) is black, no violation.

```

1      20(B)
2     /  \
3    10(B) 30(B)
4   /  \
5  5(R) 15(R)

```

Tree is balanced, all 5 rules hold.

9 Part 4 — Turning it into an order-statistics tree

A plain `std::set` is a Red-Black tree, but it can't answer "how many elements are smaller than x ?" or "what's the k -th smallest element?" in $O(\log n)$ — because a node in a plain RB tree only knows its own key and its children's pointers. It has no idea how many descendants it has.

`tree_order_statistics_node_update` fixes this by attaching one extra integer to every node: `subtree_size`.

```

1 int sizeof(Node* n) { return n ? n->size : 0; }
2
3 void updateSize(Node* n) {
4     if (n) n->size = 1 + sizeof(n->left) + sizeof(n->right);
5 }

```

Every insert, delete, or rotation calls this update on the handful of nodes it touched. Since a rotation only changes the parent/child relationship of two nodes (x and y), only their sizes need recomputing — and in a specific order:

```

1 void rotateLeft(Node* x) {
2     Node* y = x->right;
3     x->right = y->left;
4     if (y->left) y->left->parent = x;
5     y->left = x;
6     // ... parent pointer bookkeeping omitted for brevity ...
7     updateSize(x); // x's children changed -> fix x first
8     updateSize(y); // y's size depends on x's corrected size
9 }

```

Why nothing above y needs updating: a rotation doesn't add or remove any nodes from the tree — it just reshuffles which nodes are whose children. Every node outside $\{x, y, \text{the subtree that moved}\}$ still has exactly the same descendants it had before. Since `size` is purely a function of a node's current children, only x and y can possibly have a new size, and y must be recomputed *after* x because y 's children now include the corrected x .

9.1 The two payoff operations

order_of_key(q) — count of elements strictly less than q :

```

1 int order_of_key(int q) {
2     Node* x = root;
3     int count = 0;
4     while (x) {
5         if (q > x->key) {
6             count += sizeOf(x->left) + 1;
7             x = x->right;
8         } else {
9             x = x->left;
10        }
11    }
12    return count;
13 }

```

Correctness, by induction on the walk. If $q > x \rightarrow \text{key}$, then every node in x 's left subtree is $< x \rightarrow \text{key} < q$ (that's `size(left)` matches), and x itself is also $< q$ (+1 more) — so we add exactly `size(left) + 1` and recurse right, where any remaining smaller-than- q elements must live. If $q \leq x \rightarrow \text{key}$, then x and its whole right subtree are $\geq q$, contributing nothing, and we recurse left. Both branches preserve the invariant "count so far is exactly correct for everything already visited," so by induction the final count is exact. ■

find_by_order(k) — the k -th smallest element:

```

1 Node* find_by_order(int k) {
2     Node* x = root;
3     while (x) {
4         int L = sizeOf(x->left);
5         if (k < L) x = x->left;
6         else if (k == L) return x;
7         else { k -= (L + 1); x = x->right; }
8     }
9     return nullptr;
10 }

```

Both run in time proportional to the height of the tree, which we already proved is $O(\log n)$. So on paper, this gives a fully indexable, rank-queryable balanced BST at $O(\log n)$ per operation.

check it's uses here.

Video: <https://youtu.be/IWyIwLFucU4>

10 Part 5 — Counting exactly how many operations my code did

Here's the structure of the hot loop in my first (TLE'd) solution:

```

1 rep(i, 0, n) {
2     rep(j, 0, n) {
3         int g = prodTree.order_of_key({a[i]*b[j], INF});
4         int h = aTree.order_of_key({a[i], INF});
5         // ... accumulate into total ...
6     }
7     rep(j, 0, n) {
8         prodTree.insert({a[i]*b[j], prodTimer++});
9     }
10    aTree.insert({a[i], aTimer++});
11 }

```

Let's count every single operation exactly, for $n = 2000$ (the worst case, since $\sum n \leq 2000$ still allows one test case with $n = 2000$).

Operation	How many times	Which tree	Max size that tree reaches
order_of_key on prodTree	$n \times n = n^2$	prodTree	n^2
order_of_key on aTree	$n \times n = n^2$	aTree	n
insert into prodTree	$n \times n = n^2$	prodTree	n^2
insert into aTree	n	aTree	n

Total tree operations:

$$T_{\text{total}} = 2n^2 \text{ (prodTree)} + (n^2 + n) \text{ (aTree)} = 3n^2 + n$$

For $n = 2000$, $n^2 = 4,000,000$:

$$T_{\text{total}} = 3(4,000,000) + 2000 = 12,002,000 \text{ tree operations}$$

Twelve million operations isn't a large number on its own — modern CPUs execute billions of instructions per second. The important question is what each of these operations actually costs on real hardware, not just in theory.

10.1 Cost per operation, using the height bound we proved

prodTree grows up to $n^2 = 4,000,000$ entries:

$$h_{\text{prodTree}} \leq 2 \log_2(4,000,001) \approx 2 \times 21.93 \approx 43.9 \rightarrow \mathbf{44 \text{ pointer hops}}$$

aTree grows up to $n = 2000$ entries:

$$h_{\text{aTree}} \leq 2 \log_2(2001) \approx 2 \times 10.97 \approx 21.9 \rightarrow \mathbf{22 \text{ pointer hops}}$$

Total pointer traversal steps across the whole run:

$$2n^2 \text{ (prodTree ops)} \times 44 = 8,000,000 \times 44 = 352,000,000$$

$$(n^2 + n) \text{ (aTree ops)} \times 22 = 4,002,000 \times 22 \approx 88,044,000$$

$$\text{Grand total} \approx 352,000,000 + 88,044,000 = 440,044,000 \approx 4.4 \times 10^8 \text{ pointer hops}$$

10.2 Turning pointer hops into wall-clock seconds

This is where the TLE actually comes from. A pointer hop in a tree made of heap-allocated nodes isn't free the way an array index is. Every node was allocated separately by `new`, so nodes end up scattered across essentially random addresses in RAM. Walking parent $\rightarrow$ child $\rightarrow$ child means jumping between memory locations the CPU has no way to predict in advance.

Modern CPUs read memory in 64-byte cache lines and keep recently-used lines in small, extremely fast caches (L1, L2, L3). Rough costs on a typical 3 GHz machine:

- L1 cache hit: ~ 4 cycles $\approx \mathbf{1 \text{ ns}}$
- Main memory (cache miss): ~ 300 cycles $\approx \mathbf{100 \text{ ns}}$

Because each pointer hop in our 4-million-node tree lands on an essentially random address, almost every hop is a cache miss. Using the pessimistic-but-realistic 100 ns/hop figure:

$$4.4 \times 10^8 \text{ hops} \times 100 \text{ ns} = 44,000,000,000 \text{ ns} = 44 \text{ seconds}$$

On top of that, every `insert` calls `new` under the hood — $n^2 = 4,000,000$ heap allocations, at roughly 20 ns each once you include allocator bookkeeping and rebalancing overhead:

$$4,000,000 \times 20\text{ns} = 0.08 \text{ seconds}$$

— negligible compared to the 44 seconds above, but not zero.

Estimated total: ~ 44 seconds. The actual time limit is 4 seconds — roughly 11x over budget, which matches what happened on both submissions: the judge simply hit the 4000ms cutoff.

In practice not every single hop costs the full 100 ns — the top few levels of an actively-used tree tend to stay in L2/L3 cache, which is why the real run finished in some large but finite time rather than a literal 44 seconds before Codeforces terminated it. But the order of magnitude, tens of seconds needed versus 4 allowed, is enough to explain the failure on its own.

The algorithmic complexity, $O(n^2 \log n)$, was completely correct. The problem was that the constant hidden inside that $\log n$ was roughly **100x larger** than it needed to be, purely because of how the data was laid out in memory.

11 Attempt #2: swap the tree for a flat array

The fix keeps the exact same math — same comparisons, same accumulation logic — and only changes the data structure. Instead of a Red-Black tree of heap nodes, I use coordinate compression plus a **Fenwick tree** (Binary Indexed Tree) backed by one contiguous vector<int>.

First, compress every value I'll ever query into a small rank:

```
1 vi vals;
2 rep(i, 0, n) {
3     vals.pb(a[i]);
4     rep(j, 0, n) vals.pb(a[i] * b[j]);
5 }
6 uni(vals); // sort + unique
```

A tiny helper turns any real value into its compressed rank via binary search:

```
1 auto get_id = [&](int x) {
2     return lower_bound(all(vals), x) - vals.begin() + 1;
3 };
```

The Fenwick tree implementation is simple — just an array and two loops:

```
1 struct Fenwick {
2     int n;
3     vector<int> tree;
4     Fenwick(int n) : n(n), tree(n + 1, 0) {}
5
6     void add(int i, int delta) {
7         for (; i <= n; i += lsb(i)) tree[i] += delta;
8     }
9     int query(int i) {
10        int sum = 0;
11        for (; i > 0; i -= lsb(i)) sum += tree[i];
12        return sum;
13    }
14};
```

And the hot loop barely changes shape at all — order_of_key becomes query, insert becomes add:

```
1 rep(i, 0, n) {
2     rep(j, 0, n) {
3         int g = prodTree.query(get_id(a[i] * b[j]));
4         int hii = sz_prod - g;
5
6         int h = aTree.query(get_id(a[i]));
7         int hiiuu = sz_a - h;
8
9         int diff = (hii - hiiuu) % MOD;
10        if (diff < 0) diff += MOD;
11        total = (total + (diff * fact[n-2]) % MOD) % MOD;
12    }
13    rep(j, 0, n) {
14        prodTree.add(get_id(a[i] * b[j]), 1);
15        sz_prod++;
16    }
17    aTree.add(get_id(a[i]), 1);
18}
```

```

18     sz_a++;
19 }

```

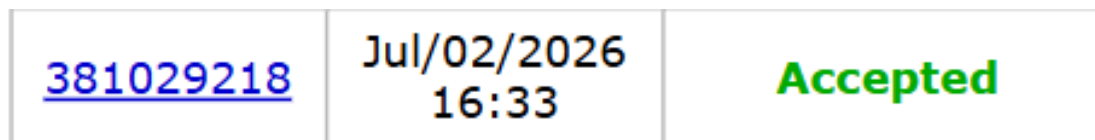
(I also fixed a subtle negative-modulo bug along the way — `diff` can go negative before the `% MOD`, so it needs an explicit `if (diff < 0) diff += MOD` guard, and I precompute factorials once instead of recomputing an inverse in the inner loop.)

This submission passed:

```

1 381029218 Accepted 3625 ms 92900 KB

```



Accepted submission

3625 ms, inside the 4000 ms limit. The algorithmic complexity didn't change from the first attempt, so the difference has to come from somewhere else.

Part 6 — Why the Fenwick tree passes: same math, different constant

The Fenwick tree does the *same number* of "logical steps" as the PBDS tree — for $n = 2000$, tree height was ≈ 44 for the big structure and ≈ 22 for the small one, and a Fenwick tree over the same number of elements needs the same $\lceil \log_2(\text{size}) \rceil$ iterations per add/query, since each step strips off the lowest set bit. The total operation count $T_{\text{total}} = 3n^2 + n \approx 12$ million and total "step" count $\approx 4.4 \times 10^8$ are essentially identical to before.

The difference is entirely in what each step *costs*, physically.

A Fenwick tree is backed by one `vector<int>` allocated once, up front, as a single contiguous block of memory. Walking it means computing `i -= lsb(i)` or `i += lsb(i)` and jumping to a new array index — and because the whole array is one block, those jumps land inside memory the CPU has *already loaded* into cache, or can *predict* and prefetch, rather than jumping to a brand-new random heap address every single time.

For $n = 2000$, the compressed value array `vals` has at most $n + n^2 = 2000 + 4,000,000 \approx 4,000,000$ entries, so each Fenwick tree (`prodTree`, `aTree`) is a `vector<int>` of a few million 4-byte integers — a few tens of megabytes. That's too big to fit entirely in L1 (usually 32–48 KB) or L2 (a few hundred KB to a few MB) cache, but:

- it fits comfortably in **L3 cache** on most modern CPUs (typically 8–32 MB), and
- even where it spills into RAM, **sequential/strided access to one contiguous array is exactly the pattern hardware prefetchers are built to detect and pre-load ahead of time.**

Redoing the same wall-clock estimate as before, but now assuming most accesses hit cache (say, an average of ~ 2 ns per step instead of ~ 100 ns, blending a mix of L1/L2/L3 hits with the occasional real RAM miss):

$$4.4 \times 10^8 \text{ steps} \times 2\text{ns} \approx 0.88 \text{ seconds}$$

That's roughly 50x faster than the ~44 second PBDS estimate, for the same number of logical steps. It also removes the other overhead the tree had: no new call on every insert (the vector for each Fenwick tree is allocated exactly once, up front), no red-black rebalancing, no rotations, no recoloring, no extra size bookkeeping through parent pointers.

The actual submission ran in 3625 ms rather than my rough 0.88s estimate — real code always carries extra constant factor from the modular arithmetic, the `lower_bound` binary searches for coordinate compression, and I/O — but it lands in the same universe as the estimate, and critically, on the correct side of the 4-second wall, instead of ~10x over it.

12 Summary

Both solutions are $O(n^2 \log n)$. Both perform roughly the same number of logical tree steps. The difference between the TLE and the accepted run came down to where the data physically lives in memory:

- A tree of individually new'd nodes is scattered across RAM, so each traversal involves multiple unpredictable cache misses, each costing on the order of 100 ns.
- A flat array backing a Fenwick tree is one contiguous block, so traversal is mostly cache hits and prefetch-friendly access patterns, each costing on the order of 1–2 ns.

Same time complexity, but a 50–100x difference in the constant factor, entirely due to memory layout. `ordered_set` and `ordered_multiset` are useful — for smaller operation counts they're simpler to write and fast enough. But once the operation count reaches into the millions in a tight loop, that per-operation constant becomes the deciding factor, and a flat array-based structure like a Fenwick tree or segment tree is usually the better choice.

Here's the minimal Fenwick tree itself, runnable and editable below. It reads n values followed by q inclusive $[l, r]$ range-sum queries (0-indexed) and prints the sum for each — try changing the array or queries and hit Run, or use the test cases underneath to check it against a couple of known answers.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Fenwick {
5      int n;
6      vector<long long> tree;
7      Fenwick(int n) : n(n), tree(n + 1, 0) {}
8
9      void add(int i, long long delta) {
10         for (++i; i <= n; i += i & (-i)) tree[i] += delta;
11     }
12     long long prefixSum(int i) {
13         long long sum = 0;
14         for (++i; i > 0; i -= i & (-i)) sum += tree[i];
15         return sum;
16     }
17 };
18
19 int main() {
20     int n;
21     cin >> n;
22     Fenwick fen(n);
23     for (int i = 0; i < n; i++) {
24         long long x; cin >> x;

```

```
25     fen.add(i, x);
26 }
27
28 int q;
29 cin >> q;
30 while (q--) {
31     int l, r; cin >> l >> r;
32     long long sum = fen.prefixSum(r) - (l ? fen.prefixSum(l - 1) : 0);
33     cout << sum << "\n";
34 }
35 }
```

Test cases**ascending array, two ranges***Input:*

```
5
1 2 3 4 5
2
0 2
1 4
```

Expected output:

```
6
14
```

includes a negative value*Input:*

```
3
10 -5 7
1
0 2
```

Expected output:

```
12
```

Run them in the browser at <https://priyanshudebnath.me/blog/pbds-vs-fenwick-cache-tle/>.